clusterd

Design, wire protocol, operations and limitations
Phase 1: heartbeat and status

7 November 2026

Abstract

clusterd pools a small number of Linux machines so that independent simulation runs can be submitted from one place and placed on whichever machine has free capacity, and so that their disks can later be combined into redundant shared storage. This document describes the system as implemented at the end of Phase 1: one agent binary that runs on every machine, a mutually authenticated TLS control channel rooted in a cluster-private certificate authority, periodic heartbeats carrying CPU, memory and disk figures, and a `clusterd status` command that shows which machines are up. It explains the reasoning behind each design choice, gives the complete wire-protocol reference and the procedure for deploying on real machines, and then sets out in detail what the current design does not do, what it does poorly, and the defects found while writing it.

Contents

1 Introduction

1.1 Purpose

clusterd bundles several modest Linux machines into one pooled system administered from a single place. Two capabilities are wanted:

- *Job pooling*: submit simulation jobs from one place and have each one placed on whichever machine has free capacity.
- *Storage pooling*: combine the disks of all machines into one storage space with parity-based redundancy ("RAID 5" semantics: losing one disk or one machine loses no data).

Three constraints shape everything else. The simulation code must run unmodified. The system is administered entirely over SSH. And it must feel like one system, not like a fleet of machines managed one by one.

The project starts as a prototype but is meant to grow into a production system in an environment where failure has a real cost. That does not change the incremental approach — a small, well-tested foundation is the stable way to start — but it raises the bar within each increment: thorough tests from the first phase, careful handling of every byte that arrives from the network, and no privilege or assumption that would be hard to walk back later.

1.2 Non-goals

clusterd does not try to make several machines look like one computer to a single unmodified process (transparent distributed shared memory, or CPU pooling across machines). That was the original request. The actual workload — CPU-bound, needing more memory than one machine has, running in a managed runtime — rules it out for any lightweight system. The technologies that really do this are either proprietary hypervisor-level products that need specialised RDMA hardware (ScaleMP, TidalScale) or research systems that are abandoned and were never fast (Kerrighed, openMosix).

The goal was therefore reframed to what is tractable without touching the simulation code: job-level parallelism (many independent runs spread across machines) plus pooled, redundant storage. One consequence must be understood by every user: *a single simulation run is still limited to the CPU and memory of the one machine it runs on.* clusterd shortens the time to finish a batch of runs; it does not make an individual run faster or let it use more memory.

1.3 Scope of this document

This document covers Phase 1 as implemented, the phases planned after it, and — in detail — the current limitations (Section **??**) and known defects (Section **??**). Whenever the text says "planned" or names a later phase, the corresponding code does not exist yet.

This is an engineering description, not a security audit or a certification. Before clusterd carries real weight in a critical environment, it needs a dedicated security and reliability review.

2 Design principles

2.1 Delegate to proven tools

Wherever a proven tool already solves a problem correctly, clusterd delegates to it. From-scratch Rust is written only for the coordination logic that is specific to this project. Table **??** lists the delegations, including those planned for later phases.

Problem	Delegated to	clusterd's own job	Phase
Encryption and mutual authentication of the control channel	rustls (ring backend)	Create a small cluster CA and one certificate per machine	1
CPU, memory and disk figures	`sysinfo` crate	Sample on every heartbeat and report to the head	1
Process supervision and start at boot	systemd	Generate unit files, enable and start them, poll their status	3
Remote authentication and first contact with a new machine	system `ssh` and `scp`	Invoke them; never reimplement SSH	3
Redundant distributed storage	SeaweedFS (`weed`)	Generate its configuration, run it under systemd, choose redundancy settings	4

Table 1: Problems delegated to existing tools.

2.2 Small, complete increments

Distributed systems do not work as one giant first commit. Each phase lands as a complete, working, tested increment rather than as part of a large, partially working system (Table **??**).

Phase	Status	Delivers
0 — workspace skeleton	Done	Cargo workspace with the three crates
1 — heartbeat and status	Done	Agent runtime, mutual TLS, live node table, `clusterd init`, `clusterd cert issue`, `clusterd status`
2 — job submission and scheduling	Not started	`clusterd run`, placement, log streaming, job persistence
3 — automated node join	Not started	One-command onboarding over SSH, systemd unit installation
4 — storage orchestration	Not started	Pooled redundant storage on SeaweedFS, FUSE mount

Table 2: Phases and their status.

2.3 One binary; the role is configuration

Every machine, the head included, runs the same command, `clusterd agent run`. Whether a machine is the head or a worker is a single field in its `config.toml`, not a different program. The head is a superset of a worker: it contributes its own CPU, memory and disk to the pool exactly like a worker, and in addition it accepts connections from workers, keeps the node table, and serves the local control socket used by the command-line interface. From Phase 2 it will also run the scheduler.

This leaves one artifact to build, copy and upgrade, and head and workers cannot drift apart in behaviour.

2.4 Pure logic kept apart from input and output

The node table — the code that decides whether a machine is up — does no networking and no sleeping. Its test-facing methods take the current time as a parameter (`record_heartbeat_at`, `snapshot_at`), so the behaviour at the timeout boundary is tested deterministically against a fabricated clock. The networking code in `agent.rs` is kept as thin wiring around such pure parts.

2.5 No dependency or state before it is needed

Phase 1 has no database. Nothing needs to survive a restart except the configuration and the certificates, and the node table correctly rebuilds itself from incoming heartbeats. SQLite (through `rusqlite`) arrives with Phase 2, which has job history and a queue worth persisting; adding it earlier would mean an unused dependency. The same reasoning rejected gRPC and QUIC (Section **??**).

The release profile enables link-time optimisation, a single code generation unit and symbol stripping, producing one compact executable.

3 Architecture

3.1 Crates

The Cargo workspace has three crates. Dependencies point in one direction only: `clusterd` depends on `clusterd-core`, which depends on `clusterd-proto`.

`clusterd-proto` The cross-machine contract: the `Message` enum and its framing. It has no opinion about the async runtime beyond the codec trait of `tokio-util`. Keeping it separate keeps the contract small, reviewable and testable in isolation, including by fuzzing.

`clusterd-core` Everything else, as a library: agent runtime, TLS and identity, node table, resource sampling, local control protocol and configuration.

`clusterd` The command-line program, built with `clap`; thin wiring over `clusterd-core`.

3.2 Modules of clusterd-core

`config` The TOML configuration (`node_id`, `node_name`, `role`, `listen_addr` or `head_addr`, heartbeat interval and timeout) and the file layout of the data directory. Validation only checks that a head has a listen address and a worker has a head address.

`identity` Generation of the cluster CA and of per-machine certificates with `rcgen`.

`tls` Builds the rustls server and client configurations for mutual TLS from PEM files, and installs the ring crypto provider once per process.

`nodes` The node table: records heartbeats and decides up or down. Pure and synchronous.

`resources` CPU, memory and disk sampling through `sysinfo`.

`agent` The runtime: the head's listener and per-connection handling, the head's own heartbeat, and the worker's connect, heartbeat and reconnect loop.

`ipc` The local protocol between the command-line program and the agent on the same machine, over a Unix-domain socket.

`error` One flat error type for the whole crate.

3.3 Runtime structure

Figure **??** shows the processes and tasks at run time.

Head. At start-up (`run_head`) the head loads its certificate, its private key and the CA certificate and builds the TLS server configuration. It creates the node table, shared between tasks behind a mutex, and spawns two background tasks: a loop that samples the head's own resources and records them directly in the table (no network round trip to itself), and the local control-socket server. It then binds the TCP listener and loops forever, accepting connections and spawning one task per connection.

Worker. A worker (`run_worker`) loops forever around one function, `connect_and_heartbeat`. Each attempt reloads the certificate files from disk, opens TCP to the head, completes the TLS handshake, sends `Hello`, waits for `HelloAck`, and then waits on two events at once: the heartbeat timer (sample resources and send a heartbeat) and incoming data (used only to notice that the connection has closed). When the attempt ends for any reason, `run_worker` waits for a back-off delay and tries again (Section **??**).

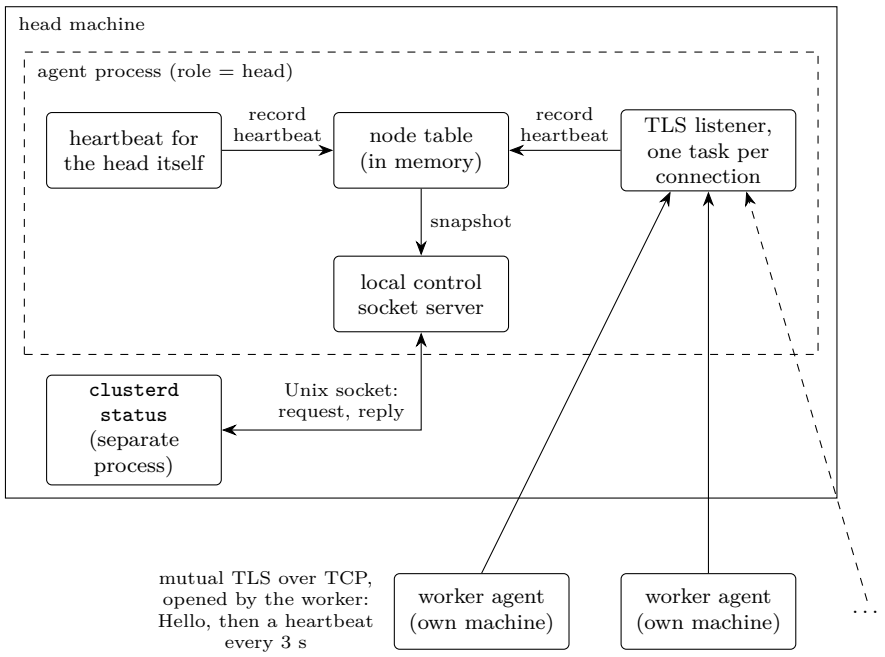


Figure 1: Run-time structure. Solid arrows into the node table are writes; the control socket only reads a snapshot. Workers always open the connection; the head never dials out.

Command-line program. `clusterd status` never touches the network. It connects to the `clusterd.sock` socket of the agent on the same machine, sends one request and prints the reply. The intended use is to SSH into the head and run commands there: the command needs no network credentials of its own, and it answers from data the head has already accumulated, with no round trip to the workers.

3.4 Concurrency model

The agent runs on the multi-threaded Tokio runtime. The only shared mutable state is the node table, behind a standard (blocking) mutex. Every critical section is a hash-map insert or a copy of the table, and the lock is never held across an `.await`, so a blocking mutex is the right tool. Each worker connection has its own task, so a slow or misbehaving connection does not hold up the others.

3.5 State and persistence

On disk there is only what `clusterd init` and `clusterd cert issue` write: the configuration and the certificate files. The node table lives in the head's memory; it is lost when the head restarts and is rebuilt as the workers reconnect. A path for a future database, `state.db`, is reserved in `config` but not used.

4 Identity and trust

4.1 Trust model

Trust is rooted in a small certificate authority created by `clusterd init --role head` and private to the cluster. Every control connection uses mutual TLS. The head requires a client certificate signed by the cluster CA; there is no anonymous mode. The worker verifies the head's certificate against the same CA. The system's trust store is never consulted. The cryptography is done by rustls with the ring backend; only TLS 1.3 is compiled in (the `tls12` feature of rustls is not enabled).

This is a deliberate choice over a shared secret: rustls does all the cryptography, and clusterd only decides who gets a signed certificate. In Phase 3, delivering a certificate to a new machine will ride on SSH, which is already an authenticated channel to that machine — one trust mechanism bootstrapping the other.

4.2 Certificates

Why every machine has the same name. Authorisation comes entirely from "this certificate was signed by our CA", not from matching a host name. A fixed subject alternative name (`clusterd-node`, the constant `NODE_SAN`) avoids any dependence on DNS or reverse DNS on a small local network, and lets a

Certificate	Properties
Cluster CA	Self-signed; common name “clusterd cluster CA”; marked as a CA with no limit on path length; ECDSA P-256 key. Created once per cluster, on the head.
Machine certificate (head and each worker)	Signed by the cluster CA; common name set to the machine’s name; one subject alternative name, `DNS:clusterd-node`, identical for every machine; ECDSA P-256 key; no key-usage or extended-key-usage extension.

Table 3: Certificates issued by clusterd. Both kinds use the `rcgen` default validity, 1 January 1975 to 1 January 4096.

worker reach the head by IP address while still performing standard TLS server-name verification: the worker always asks for `clusterd-node`. The price of this choice is discussed in Section **??**.

How a certificate is issued. `rcgen` cannot load a signing handle directly from a PEM file. `issue_node_cert` therefore rebuilds the CA’s parameters from its certificate and signs them again with the original CA key, which gives an issuer handle; the new machine certificate is then signed with the original CA key. The rebuilt CA certificate is only used in memory and is never written out. The integration test confirms that rustls accepts certificates issued this way.

4.3 Three identifiers

A machine is known by three things, summarised in Table **??**.

Identifier	Origin	Used for
`node_id`	UUID version 7, generated by `clusterd init`, stored in `config.toml`	Key of the node table; stable across restarts and renames
`node_name`	`--node-name` of `clusterd init`, stored in `config.toml`	Display only; may be changed without changing the identity
Certificate	`clusterd init` (head) or `clusterd cert issue` (workers); its common name comes from `--name`	Admission to the cluster: any valid certificate is admitted

Table 4: The three identifiers of a machine.

These three are *not* bound to each other. The head admits any connection

presenting a valid certificate and then believes whatever `node_id` and `node_name` the `Hello` message announces; the certificate's common name is never compared with the announced name. Section **??** discusses the consequences.

4.4 Key files and privileges

Private keys (`ca.key`, `node.key`) are created with mode `0600` from their first byte (`write_secret` in the command-line program), so there is no window during which a key sits on disk with wider permissions. Certificates and the configuration are not secret and use default permissions.

The agent runs without privileges: `sysinfo` reads `/proc` and filesystem statistics without root. Root will be needed in Phase 3 (installing systemd units) and Phase 4 (managing FUSE mounts). The plan is to grant narrow Linux capabilities, or to use a helper with a tightly scoped `sudo` rule, rather than run the whole agent as root.

4.5 Untrusted input

The message decoder is the one function that parses bytes fully controlled by the other side of a connection. Its input is bounded to 16 MiB, both by the framing codec and again inside `Message::decode`; it returns an error rather than panicking; and it is covered by adversarial tests: every truncation of a valid message, garbage bytes, empty input, an oversized buffer, an oversized length prefix, and a property-based test that feeds it arbitrary byte strings of up to 4 KiB. Frames are parsed only after the TLS handshake has succeeded, so a peer without a valid certificate never reaches the decoder.

5 Wire protocol

The protocol is defined in `crates/clusterd-proto/src/message.rs`.

5.1 Transport and framing

From the bottom up, a connection consists of:

1. one TCP connection per worker, opened by the worker;
2. mutual TLS 1.3 over it;
3. length-delimited frames: a 4-byte big-endian length followed by the payload, at most 16 MiB (`MAX_FRAME_LEN`). Both sides build the codec through the single function `clusterd_proto::message_codec`, so their limits cannot differ;
4. one `Message` per frame, encoded with bincode 1.3 in its default configuration: little-endian fixed-width integers, a 4-byte tag for the enum variant, an 8-byte length before every string and list. A heartbeat is on the order of a hundred bytes.

Why not gRPC or QUIC. gRPC (through `tonic`) brings a large dependency tree — `prost`, `hyper`, `h2`, `tower` — built for problems of many services behind load balancers, which this system does not have. QUIC's strengths are surviving packet loss and changing network paths on unreliable networks; on a trusted local network that complexity buys nothing.

Why the worker opens the connection. Only the workers need to know an address, the head's. The head never has to track or dial worker addresses, so a worker whose IP address changes (for example on a DHCP renewal) simply reconnects. The converse is that the head's address must stay fixed (Section **??**).

5.2 Connection life cycle

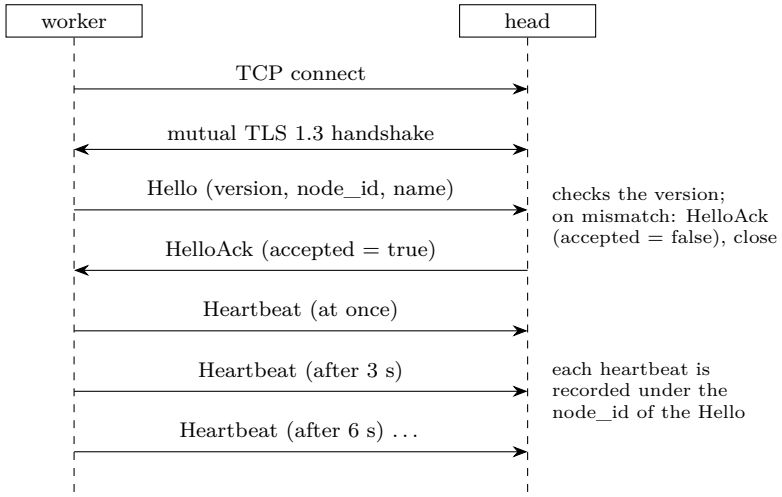


Figure 2: A successful connection. After HelloAck the head sends nothing more; the channel is one-way.

Figure **??** shows a successful connection. Step by step:

1. The worker opens TCP to `head_addr` and completes the TLS handshake. The worker checks that the head's certificate is signed by the cluster CA and carries the name `clusterd-node`; the head checks that the worker's certificate is signed by the cluster CA.

2. The worker sends `Hello` as the first frame.

3. The head compares `protocol_version` with its own. On a mismatch it sends, on a best-effort basis, `HelloAck` with `accepted` set to false and a `reason`, then closes the connection. Otherwise it replies with `accepted` set to true. Any other first message closes the connection.

4. The worker starts heartbeating: the first heartbeat immediately, then one every `heartbeat_interval_secs` (3 s by default). If the worker falls behind, missed ticks are delayed rather than sent in a burst.

5. The head records each heartbeat in the node table under the `node_id` given in `Hello`. A heartbeat carrying a different `node_id` is logged and dropped. Other message types are logged and ignored. A frame that cannot be decoded closes the connection.

6. The connection ends when either side closes it or an error occurs. There is no goodbye message: from the head's point of view a closed connection and a silently stalled one look the same, and both are handled by the same timeout (Section **??**).

After `HelloAck` the head sends nothing more. The worker reads only to notice that the connection has closed, and ignores any frame it receives without decoding it.

5.3 Message reference

Phase 1 has three message types. The head's own heartbeat has the same content as `Heartbeat` but is recorded in-process, without going through the network.

5.4 Versioning

`PROTOCOL_VERSION` (currently 1) is incremented on every change to `Message` that breaks the encoding. Because it is checked during the handshake and must match exactly, a version mismatch fails loudly and immediately instead of silently misreading bytes on either side.

Adding new variants at the *end* of the enum does not change how the existing variants are encoded, so the messages planned for Phases 2 and 3 can be added without breaking the Phase 1 ones. This compatibility is only partial; see Section **??**.

5.5 Local control protocol

The command-line program and the agent on the same machine talk over a Unix-domain socket, `clusterd.sock` in the data directory, using `LocalRequest` and `LocalResponse` from `ipc.rs`. Framing is the same length-delimited kind, with a 4 MiB limit, carrying bincode. It is deliberately a different enum and a different codec instance from the cross-machine protocol: this traffic never leaves the machine, so it has no versioning or authentication concerns of its own — access is governed by filesystem permissions on the socket. Each connection carries exactly one request and one reply, and only the head runs the server.

Field	Type	Meaning
Hello — worker to head, first frame only		
`protocol_version`	`u16`	The sender's `PROTOCOL_VERSION`; must equal the head's.
`node_id`	UUID v7	Stable identity from `config.toml`. Every heartbeat on this connection is filed under it.
`node_name`	string	Human-readable name from `config.toml`; display only.
`cert_fingerprint`	string	Reserved for audit logging. The current worker sends an empty string and the head ignores the field.
HelloAck — head to worker, reply to Hello		
`protocol_version`	`u16`	The head's own version, so a rejected worker can log what was expected.
`accepted`	`bool`	False only on a version mismatch.
`reason`	optional string	Explanation for a human reading the logs; not meant for program logic.
Heartbeat — worker to head, on a timer		
`node_id`	UUID	Must equal the `node_id` of this connection's `Hello`.
`cpu_free_pct`	`f32`	0 to 100; see Section `??`.
`mem_total_bytes`	`u64`	Physical memory.
`mem_free_bytes`	`u64`	Memory available to new work.
`disk_free_bytes`	`u64`	Free space on the filesystem holding the data directory.
`running_jobs`	list of UUID	Always empty in Phase 1.
`agent_version`	string	Package version of the sending binary.
`timestamp`	system time	The sender's wall clock when sampling. Stored, but neither shown nor used for the timeout.

Table 5: Phase 1 messages.

Request	Reply	Notes
`Status`	`Status` with one `NodeStatus` per known machine, or `Error` with a message	Backs `clusterd status`; answered from the node table.

Table 6: Local control requests.

A `NodeStatus` contains `node_id`, `node_name`, `up`, `last_seen_secs_ago`, the four resource figures, the number of running jobs and `agent_version`; entries are sorted by name.

At start-up the server deletes any file already at the socket path, so a socket left behind by a crash does not prevent it from binding. This has a side effect described as defect D2 in Section **??**.

6 Failure detection and recovery

6.1 The up/down rule

For each `node_id` the head stores the instant at which it *received* the latest heartbeat — not the time the sender claims to have sent it. This instant comes from the head's monotonic clock (`std::time::Instant`), so clock differences between machines and adjustments of the wall clock have no effect on the decision. When the table is queried, a machine is *up* if and only if

$$now - last\ received \leq \texttt{heartbeat_timeout_secs}$$

(15 s by default; the boundary is inclusive and is tested exactly at 15 s and 16 s). Otherwise `clusterd status` reports it as `DOWN`.

6.2 Detection latency and tolerance

Let I be the heartbeat interval and T the timeout. A worker that stops at time t sent its last heartbeat at some time in $(t - I,\, t\,]$, so it is reported down between $T - I$ and T after it stopped — between 12 and 15 seconds with the defaults — plus network delay. In the other direction, a machine that is alive but whose heartbeats are lost or late is reported down only after no heartbeat has arrived for T seconds; with the defaults it survives about four consecutive lost heartbeats.

The head's own entry is fed in-process and cannot go down while the head agent is running.

6.3 Reconnection

All retry policy lives in one place, `run_worker`. After any failure the worker waits and tries again. The wait starts at 1 s and doubles after every failed attempt up to 30 s (1, 2, 4, 8, 16, 30, 30, … s). If the attempt that just ended had lasted more than 10 s — that is, the connection had been healthy — the wait first goes back to 1 s. Every kind of failure is retried this way: an unreachable head, a rejected certificate, a rejected version, missing certificate files. The worker never exits on its own.

6.4 Restart of the head

When the head agent restarts, its node table starts empty. The head's own entry reappears immediately; each worker reappears when it reconnects. If only the head *process* stopped, the operating system closes its connections, so the workers notice at once and reconnect within their back-off delay (at most 30 s after the head is ready). If the head *machine* disappeared, the workers can take much longer to notice (Section **??**).

7 Resource figures

Each heartbeat carries a fresh sample taken by `ResourceSampler` (`resources.rs`) on the machine that sends it.

Field	How it is measured
`cpu_free_pct`	100 minus the average usage over all cores since the previous sample, as computed by `sysinfo`, clamped to 0–100.
`mem_total_bytes`	Total physical memory.
`mem_free_bytes`	*Available* memory (`MemAvailable` on Linux): memory that can be given to new work without swapping, including reclaimable cache. This is the right figure for placement decisions, rather than the kernel's strict "free" memory.
`disk_free_bytes`	Space available on the filesystem whose mount point is the longest prefix of the data-directory path. In Phase 4 this will be the dedicated storage volume instead.
`running_jobs`	Always empty until Phase 2.

Table 7: How the resource figures are obtained.

8 Operations

8.1 Command-line interface

All commands accept the global option `--data-dir`; its default is `./clusterd-data`. Use an absolute path instead: see defect D1.

```
clusterd init --node-name N --role head --listen IP:PORT
clusterd init --node-name N --role worker --head IP:PORT
clusterd cert issue --name N --out DIR
clusterd agent run
clusterd status
```

init (head) Creates the data directory, writes `config.toml` with a new `node_id`, creates the cluster CA and the head's own certificate, and prints the next steps.

init (worker) Writes `config.toml` only. The worker's certificate files are copied in afterwards.

cert issue Run against the head's data directory. Writes `ca.crt`, `node.crt` and `node.key` for a new machine into `DIR`.

agent run Runs the agent in the foreground.

status Run on the head. Prints one line per known machine with the columns NAME, UP, CPU FREE, MEM TOTAL, MEM FREE, DISK FREE, JOBS, VERSION and LAST SEEN.

Addresses are parsed as a numeric IP address and port; host names are not accepted.

8.2 Data directory

File	Present on	Content
`config.toml`	every machine	Role, `node_id`, name, address, heartbeat timing
`ca.crt`	every machine	Cluster CA certificate
`ca.key`	head only	Cluster CA private key, mode `0600`
`node.crt`	every machine	This machine's certificate, signed by the cluster CA
`node.key`	every machine	This machine's private key, mode `0600`
`clusterd.sock`	head only	Local control socket, created while the agent runs

Table 8: Files in the data directory.

A worker's `config.toml` looks like this:

```
node_id = "01a1b7cc-1e8a-7d92-9525-907b9c27ba73"
node_name = "worker-1"
role = "worker"
head_addr = "192.168.1.10:7700"
heartbeat_interval_secs = 3
heartbeat_timeout_secs = 15
```

A head has `listen_addr` instead of `head_addr`. The interval is used by the machine that sends heartbeats; the timeout is used only by the head, so the timeout in a worker's file has no effect. Once systemd packaging exists (Phase 3), the data directory should move to `/var/lib/clusterd`.

8.3 Logs

The log level is set with the `RUST_LOG` environment variable (default `info`). On Linux, if the systemd journal is reachable — which is the case on any systemd distribution, including in an interactive SSH session — logs go *only* to the journal, under the name of the executable:

```
journalctl -t clusterd -f
```

Elsewhere (macOS, containers without a journal) logs go to standard error. Command output, such as the status table, always goes to standard output.

8.4 Local smoke test

`bin/dev-smoke-test.sh` is a manual, human-readable demonstration on one machine. It builds the debug binary, creates fresh data directories under `data/smoke-test` in the repository, starts a head on `127.0.0.1:17700` and two workers as separate processes, waits 5 s and prints `status` (all three up), stops one worker, waits 17 s and prints `status` again (that worker `DOWN`, the others up). It proves the logic, not real networking between hosts.

8.5 Deploying on real machines

The smoke test and the automated integration test prove the logic. Only separate hosts prove that real TCP, real TLS between different machines and real `sysinfo` readings on Ubuntu behave as expected. The procedure below uses Ubuntu Server virtual machines.

Building the binary. The development Mac has no Linux cross-compilation toolchain: ring, the crypto backend, needs a C cross-compiler for Linux, and getting one that matches the target machines' C library is a project of its own. The reliable path is to build natively on one of the Ubuntu machines and copy the resulting executable to the others. Any machine can serve as build machine, including the head.

1. Copy the source to the build machine. From the Mac:

```
cd /Users/paul/myFiles/PROJECTS/PRIO01_Minimal_Distibuted_System
rsync -avz --exclude target --exclude data --exclude log \
  ./ <user>@<build-vm-ip>:~/clusterd/
```

Pushing the repository to a Git remote and cloning it on the machine works equally well.

2. Install Rust and a C toolchain on the build machine.

```
ssh <user>@<build-vm-ip>
sudo apt update && sudo apt install -y build-essential pkg-config
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs \
  | sh -s -- -y
source "$HOME/.cargo/env"
```

3. Build the release binary.

```
cd ~/clusterd
cargo build --release -p clusterd
ls -la target/release/clusterd
```

`target/release/clusterd` is the only file the other machines need.

4. Copy the binary to every other machine.

```
for host in <worker-vm-1-ip> <worker-vm-2-ip>; do
  scp target/release/clusterd <user>@$host:~/clusterd
done
```

No Rust toolchain is needed on the workers.

5. Set up the head. On the machine chosen as head:

```
./clusterd --data-dir ~/clusterd-data init \
  --node-name head-1 --role head --listen 0.0.0.0:7700
```

This creates the cluster CA and the head's certificate. Note the head's IP address; workers need it. If `ufw` is active, open the port with `sudo ufw allow 7700/tcp`.

6. Issue a certificate for each worker. Still on the head:

```
./clusterd --data-dir ~/clusterd-data cert issue \
  --name worker-1 --out ~/worker-1-certs
./clusterd --data-dir ~/clusterd-data cert issue \
  --name worker-2 --out ~/worker-2-certs
scp ~/worker-1-certs/* <user>@<worker-vm-1-ip>:~/worker-1-certs/
scp ~/worker-2-certs/* <user>@<worker-vm-2-ip>:~/worker-2-certs/
```

7. Set up each worker. On each worker, with its own name and directory:

```
./clusterd --data-dir ~/clusterd-data init \
  --node-name worker-1 --role worker --head <head-vm-ip>:7700
cp ~/worker-1-certs/* ~/clusterd-data/
```

Run `init` on each machine itself, after any cloning of virtual machines; a copied data directory duplicates the `node_id` (Section **??**).

8. Start the agents. Until Phase 3 installs systemd services, run the agents under `tmux` or `screen` so they survive the end of the SSH session. On the head and on every worker:

```
RUST_LOG=info ./clusterd --data-dir ~/clusterd-data agent run
```

The terminal stays silent: logs go to the journal. Follow them from a second session with `journalctl -t clusterd -f`.

9. Check the pool from the head. In another session on the head:

```
./clusterd --data-dir ~/clusterd-data status
```

Every machine should be listed as up, with live figures.

10. Check down detection. Stop the agent on one worker (Ctrl-C or `kill`), wait about 15 s, and run `status` again on the head. That worker should be `DOWN`; the others stay up.

8.6 Troubleshooting

- **No log output at all.** On Linux the logs are in the journal: `journalctl -t clusterd`.
- **status fails with "is clusterd agent run running on this host?".** Either the agent is not running on this machine; or `--data-dir` differs from the one the agent was started with; or this is a worker (only the head serves `status`); or the data-directory path is too long for a Unix socket (defect D4); or a second agent was started on the same data directory and removed the socket (defect D2) — restart the running agent.
- **A worker never appears in the status.** Check that the head's firewall accepts TCP on the `--listen` port, and that the worker can reach the head: run `nc -zv <head-vm-ip> 7700` on the worker.
- **Logs show only "error=io error".** This is how certificate problems appear (defect D3). Check that `ca.crt`, `node.crt` and `node.key` in the worker's data directory are the files `cert issue` produced on *this* cluster's head. If the head was initialised again, it has a new CA and every worker needs new certificates.
- **"head rejected connection: protocol version mismatch".** Head and worker run binaries with different protocol versions; copy the same binary everywhere.
- **DISK FREE shows 0.** The data directory was given as a relative path (defect D1). Use an absolute path.
- **A removed or re-initialised machine is still listed as DOWN.** The node table never forgets an entry (Section **??**). Restart the head agent to clear it.

9 Verification

As of 7 November 2026, the following commands all succeed cleanly:

- `cargo build --workspace`;
- `cargo test --workspace`: 29 tests, all passing;
- `cargo clippy --workspace --all-targets`: no warnings;
- `cargo fmt --all -- --check`: no differences.

The repository has no continuous-integration configuration; these commands are run by hand.

Suite	Tests	What it covers
`clusterd-proto` unit	9	Round trip of every variant; rejection of every truncation of a heartbeat, of garbage, of empty input and of oversized input; framed round trip over an in-memory stream; rejection of an oversized length prefix by the codec; readable error messages; property-based test that decoding arbitrary bytes never panics.
`clusterd-core` unit	19	Configuration round trip and validation; CA and certificate generation, and that rustls can load the result; building matching server and client TLS configurations; readable errors for missing or empty PEM files; node-table up/down at and past the timeout boundary, reset by a later heartbeat, sorting; resource figures within range.
Integration (`tests/integration.rs`)	1	A real head and a real worker over localhost TCP and TLS, with a 1 s interval and 2 s timeout: the worker joins and both are up; the worker is stopped and flips to down while the head stays up.

Table 9: Automated tests.

In addition, `bin/dev-smoke-test.sh` demonstrates the same scenario with three processes on one machine, and the procedure of the previous section validates it on separate machines. The gaps in this coverage are listed in Section **??**.

10 Limitations

This section lists what the current design does not do or does poorly. For each item it states the consequence and what would address it. Items that are plain bugs, rather than design limits, are listed separately in Section **??**.

10.1 Scope of Phase 1

Monitoring only. Phase 1 observes the pool; it does not use it. No job can be submitted, placed or run; `running_jobs` is always empty and the JOBS column is always 0.

Manual onboarding. Adding a worker takes several manual steps on two machines: issue a certificate on the head, copy three files, run `init` on the worker, start the agent. Phase 3's `clusterd join` is meant to reduce this to one command run on the head.

No storage pooling. Nothing in Phase 1 touches storage; disk space is only reported.

No service management and no orderly shutdown. The agent is a foreground process: it does not restart after a crash and does not start at boot. It handles no signals; Ctrl-C or `SIGTERM` end it immediately. No work is lost today, but the control socket file is left behind (it is removed at the next start). Phase 3 installs systemd units; orderly shutdown will matter from Phase 2, when a stopping worker has jobs running.

One run per machine, by design. As stated in the introduction, an individual run is limited to the resources of one machine. This is a permanent property of the design, not a temporary gap.

Small scale, one local network. The design assumes a handful to a few tens of machines on one trusted local network: all heartbeats converge on the head, through one mutex, over one connection per worker. It has not been designed or measured for wide-area networks or for hundreds of machines; the automated and scripted tests use at most three.

10.2 Availability and fault tolerance

The head is a single point of failure. Without the head there is no status, and in later phases there will be no scheduling and no storage master either; workers just keep trying to reconnect. Nothing on the head is replicated. This is accepted for now. For storage, SeaweedFS supports multiple masters with Raft natively, so that part is a bounded upgrade later; making the scheduler redundant is a separate, harder problem.

The head's address must be fixed and numeric. Each worker stores the head's IP address and port; host names are not accepted. If the head's address changes, every worker's `config.toml` must be edited. Give the head a static address or a DHCP reservation.

Workers are slow to notice a head machine that disappears. After `HelloAck` the head never sends anything, and no TCP keep-alive is configured. If the head *process* stops, its operating system closes the connections and workers notice at once. But if the head *machine* loses power or network, a worker keeps writing heartbeats into its socket buffer and only learns that the connection is dead when TCP gives up retransmitting — about 15 minutes with Linux defaults. If the head comes back earlier, its kernel answers the next retransmission with a reset; since retransmissions back off exponentially (up to two minutes apart on Linux), a worker can still stay absent from `status` for minutes after the head is back. An application-level reply from the head, or TCP keep-alive, would bound this to a few heartbeat intervals.

The head never closes a silent connection. The same reasoning in the other direction: the head only reads. If a worker machine vanishes, the head's read on that connection waits forever. The machine is correctly reported `DOWN` after the timeout, but the task, socket and TLS state of the dead connection remain until the head restarts. Each such event leaks one connection. A read timeout of a few heartbeat intervals would fix this.

The node table never forgets. There is no command to remove a machine. A decommissioned machine, or one whose data directory was re-initialised (which creates a new `node_id`), stays in `status` as `DOWN` until the head restarts.

Duplicate identities are not detected. If a virtual machine is cloned after `init`, both copies share the same `node_id` (and the same certificate). Their heartbeats overwrite each other in one row, which then alternates between the two machines' figures, and the pool appears smaller than it is. Two machines may also carry the same `node_name`; nothing checks that names are unique.

Failure detection is timeout-only and binary. "Down" means only "no heartbeat received for 15 s". A crashed machine, a network partition and a machine so overloaded that its agent misses heartbeats all look the same. In Phase 2 a false "down" could cause a job to be started twice; the scheduler will have to treat "down" as "unknown", not as "dead".

10.3 Security

Machine identity is self-declared. The head admits any connection with a valid cluster certificate and then takes `node_id` and `node_name` from the `Hello` message as given. The check on later heartbeats only stops a connection from switching identity midway. Consequently, anyone holding any one valid cluster certificate can register as any machine: overwrite another machine's row with false figures, imitate the head's own `node_id`, or open many connections with invented identifiers to fill the table. The `cert_fingerprint` field, intended for this kind of check, is sent empty and ignored. The fix is to bind identity to the

certificate — for example, put the `node_id` in the certificate at issue time and reject a `Hello` that does not match it.

Head and worker certificates are interchangeable. Every machine certificate carries the same name, `clusterd-node`, and no extended-key-usage restriction, so any worker's certificate is also a valid *head* certificate. Someone who has a worker's private key and can divert traffic on the local network (for example by ARP spoofing) can impersonate the head to the other workers. Today the impact is limited to monitoring data. From Phase 2, when the head sends jobs, impersonating the head would mean running arbitrary commands on every worker. Before Phase 2 the head needs a distinct identity (a different name or usage in its certificate) that workers require.

No revocation and no expiry. Certificates are valid from 1975 to 4096, and there is no revocation list. A certificate, once issued, admits its holder for good. The only way to exclude a lost or compromised machine is to create a new CA and reissue every certificate. Short-lived certificates renewed automatically, or a revocation list checked by the head, would address this; Phase 3's automated join is the natural place.

The CA key is online and unprotected beyond file permissions. `ca.key` sits unencrypted on the head, which is also a working machine of the pool. Whoever obtains it can issue certificates at will; compromising the head therefore compromises the whole cluster. The CA is also unconstrained (no limit on path length, no name constraints). Moving the CA key offline, or encrypting it with a passphrase needed only when issuing, would reduce this.

Re-running init overwrites an identity without warning. On the head, a second `init` silently replaces the CA, the head's certificate and its `node_id`: every worker is then rejected until it receives a new certificate. On a worker, it changes the `node_id`. `init` should refuse to overwrite an existing data directory unless forced.

The listener has no limits. There is no limit on the number of simultaneous connections, and no timeout on the TLS handshake or on the arrival of `Hello`. Anyone who can reach the port can open connections and leave them idle, each holding a task and a file descriptor. When the process runs out of file descriptors, accepting fails — and an accept failure currently stops the head (defect D5). A peer with a valid certificate can additionally send frames of up to 16 MiB, far beyond the roughly hundred bytes a heartbeat needs. Handshake and first-message timeouts, a connection limit and a much smaller frame limit would close this.

Reported figures are trusted as given. The head does not check the figures it receives: a CPU percentage outside 0–100 or not a number, more free memory than total memory, and so on, are stored and displayed as they are. This matters from Phase 2, when the scheduler will base decisions on them.

Local socket permissions are implicit. `clusterd.sock` and the data directory get whatever permissions the process's umask produces; clusterd sets none explicitly. Anyone who can connect to the socket can read the status. That is harmless today, but Phase 2 commands such as `run` over the same socket would need explicit permissions or a peer-credential check.

Not audited. None of this has had an independent security review.

10.4 Protocol evolution

Upgrades must be done all at once. The protocol version must match exactly, and there is no negotiation of capabilities. Whenever `PROTOCOL_VERSION` changes, the head and all workers must be upgraded together; a rolling upgrade is impossible.

"Additive" changes are compatible in one direction only. bincode is not self-describing: adding, removing or reordering a field of an existing message always breaks the encoding. Appending a new message type is safe only if the head is upgraded first, because an older head that receives a type it does not know fails to decode the frame and closes the connection. (An older worker ignores everything it receives after the handshake, so it does not mind.) Note also that bincode's default configuration accepts trailing bytes after a valid message, so decoding is not a strict validation of the frame.

The channel is one-way today. The head sends nothing after `HelloAck`, and the worker does not decode what it receives. Phase 2 needs the head to send jobs, so the worker side will have to start decoding messages, and both sides will have to handle unknown types gracefully.

10.5 Accuracy of the resource figures

The first sample after each connection is wrong. CPU usage is measured as a difference between two samples, and the sampler is created anew on every connection. The first heartbeat, sent immediately after the handshake, therefore always reports the CPU as 100 % free, even on a fully loaded machine. A scheduler acting on that heartbeat would choose badly. The sampler should be created once per process and primed before the first heartbeat.

One CPU percentage is not comparable between machines. The figure is an average over all cores, and the core count is not reported. “50 % free” means one core on a two-core machine and eight cores on a sixteen-core one. The scheduler will need the number of cores, and probably the load average, in the heartbeat — a protocol change.

Disk figure. The disk figure is the free space of the filesystem holding the data directory, which may have nothing to do with the space the pool will use. It is wrong (zero) with a relative path (defect D1).

Timestamps. The sender’s timestamp is stored but neither shown nor used, and “last seen” is rounded down to whole seconds.

Linux only. Only Linux is a target platform. On macOS, used for the local smoke test, `MEM FREE` was observed as 0; the smoke test there validates the logic, not the figures.

10.6 Operability

Logs are not where people look. On Linux, logs go only to the journal (when it is reachable), even when the agent is run by hand in a terminal, which then stays silent. Step 8 of the deployment procedure is the first place where this surprises. Writing to standard error as well when attached to a terminal would avoid the surprise.

Misconfiguration does not fail fast. A worker with missing or wrong certificate files, or a wrong head address, does not stop: it retries for ever with back-off and logs a warning each time — and the warning does not say why (defect D3).

status works only on the head. On a worker, `clusterd status` fails with a message suggesting the agent is not running, although it is.

Configuration values are not range-checked. Interval and timeout are accepted as written in `config.toml`. An interval of 0 makes the agent panic (defect D6). A timeout smaller than or close to the interval makes machines flap between up and down. The two values live on different machines (interval on each sender, timeout on the head), and nothing checks that they are consistent across the cluster.

The head reads its certificates once. A worker reloads its certificate files on every connection attempt, so new files take effect at the next reconnection. The head loads them only at start-up; changing them requires a restart.

The binary is not static. The executable is self-contained apart from the system C library (glibc). It must be built against a glibc no newer than that of the oldest target machine, which is why it is built natively on one of the target machines. A static (musl) build or cross-compilation from the Mac would remove this constraint later.

No health or metrics interface. `clusterd status` is the only view into the system. There is no machine-readable output, no health endpoint and no metrics export.

10.7 Testing

Fixed ports. The integration test uses port 18700 and the smoke test port 17700; both fail if the port is in use.

Untested paths. No automated test covers reconnection and back-off, rejection of a version mismatch end to end, rejection of a certificate from another CA, identity spoofing, half-open connections, or behaviour across separate machines. The defects in Section **??** all lie in these untested areas.

Timing-dependent scripts. The smoke test relies on fixed sleeps and can give false results on a heavily loaded machine. The integration test polls with a deadline instead, which is more robust.

No continuous integration. Tests are run by hand; nothing prevents a regression from being committed.

11 Known defects

These defects were found while writing this document. D1 to D4 were reproduced by running the binary on 7 November 2026 (macOS development machine); D5 to D7 follow from reading the code and standard Unix behaviour. None is fixed yet.

D1. Disk space is reported as zero with a relative data directory.

Symptom DISK FREE shows `0.0B`, including with the default `--data-dir ./clusterd-data`.

Cause `disk_free_bytes_for` (`resources.rs`) compares the data-directory path with each mount point using `Path::starts_with`, which compares path components. A relative path never starts with an absolute mount point, so no filesystem matches and the function falls back to 0.

Workaround Pass an absolute `--data-dir`. In the deployment procedure, `~/clusterd-data` is expanded by the shell into an absolute path, so it is not affected.

Fix Make the data-directory path absolute (`canonicalize`) once at start-up.

D2. A second agent on the same data directory breaks status for the first.

Symptom After an accidental second `clusterd agent run` on the head's data directory, which exits with "Address already in use", `status` fails with "connection refused" although the first agent is still running.

Cause The control-socket server removes any existing file at the socket path before binding (meant for files left by a crash), and it starts before the TCP listener. The second agent deletes the live socket, creates its own, then fails to bind the TCP port and exits. The first agent keeps running without a reachable socket.

Workaround Restart the running agent.

Fix Before deleting, try to connect to the existing socket and refuse to start if something answers; or take an exclusive lock on the data directory at start-up.

D3. Agent logs drop the cause of errors.

Symptom A worker presenting a certificate from another CA produces only `connection ended with error ... error=io error` on the head and `lost connection to head error=io error` on the worker. A control-socket failure is logged as `local control socket exited error=io error`.

Cause Errors are logged with the display text of the top-level error only, and the text of the `Io` variant is just "io error"; the underlying cause (here, the TLS alert) is not printed. Command-line commands are not affected: they print the full chain of causes.

Fix Log the full chain of causes, or include the cause in the message of the `Io` variant.

D4. A long data-directory path silently disables status.

Symptom The head runs and workers connect, but `status` fails with "path must be shorter than SUN_LEN". The agent's log only says "io error" (D3).

Cause A Unix socket path is limited to about 104 bytes on macOS and 108 on Linux. When binding fails, the error is logged and the head carries on without a control socket.

Fix Check the length at `init` and at start-up and fail with a clear message, or place the socket under a short directory such as `/run/clusterd`.

D5. A failed accept stops the head.

Cause In `run_head`, an error from `accept` is propagated out of the accept loop, which ends the agent. Accept can fail transiently, notably when the process runs out of file descriptors — which, given the unlimited listener (Section **??**), anyone who can reach the port can provoke.

Fix Log the error, pause briefly and keep accepting; limit the number of connections.

D6. A heartbeat interval of zero panics.

Cause The interval is passed to `tokio::time::interval`, which panics on a zero period, and the configuration does not reject it. On a worker the process ends with a panic. On the head, only the task that records the head's own heartbeat dies: the head's row never appears, while the agent otherwise keeps running.

Fix Validate in `Config::validate`: interval at least 1 s, timeout clearly larger than the interval.

D7. Key-file permissions are not enforced when a file already exists.

Cause `write_secret` asks for mode `0600` when opening the file, but on Unix that mode only applies when the file is created. If `node.key` or `ca.key` already exists with wider permissions (for example when `init` or `cert issue` is run again on the same directory), the new key is written with the old, wider permissions.

Fix Set the permissions explicitly after opening, or write to a new temporary file and rename it into place.

12 Roadmap

12.1 Phase 2: job submission and scheduling

Planned: `clusterd run` to submit a job from the head, placement on a machine with free capacity, streaming of the job's output back to the head, and job history and queue persisted in SQLite (`state.db`). New messages: `JobAssign`, `JobCancel`, `JobStarted`, `JobLogChunk`, `JobResult`, appended to the enum.

Phase 2 turns "impersonate a machine" into "run code on the cluster", so several limitations above become prerequisites rather than improvements. In order of importance:

1. Bind machine identity to the certificate (Section **??**).

2. Give the head a certificate identity that workers' certificates cannot provide, and have workers require it.

3. Bound the listener: handshake and first-message timeouts, connection limit, read timeout on silent connections, smaller frame limit, and a non-fatal accept loop (D5).

4. Validate reported figures; add the core count (and probably the load average) to `Heartbeat`, with a protocol version change.

5. Make the worker decode messages after the handshake and handle unknown types gracefully; add an application-level reply so that workers detect a vanished head quickly.

6. Fix D1 to D4, D6 and D7; add automated tests for the untested paths of Section **??**; set up continuous integration.

12.2 Phase 3: automated join

Planned: `clusterd join`, run on the head, which uses `ssh` and `scp` to copy the binary and a fresh certificate to a new machine, initialise it, and install and start a systemd unit. Node registration, implicit today through `Hello`, will be formalised with `NodeJoin` and `NodeJoinAck` messages. This phase needs root on the target machines; the aim is narrow capabilities or a scoped helper rather than a root agent. It is also the natural place for removing a machine, revoking or renewing certificates, and refusing to overwrite an existing data directory.

12.3 Phase 4: storage orchestration

Planned: generate SeaweedFS configuration, run its master and volume servers under systemd, and mount the pooled storage with FUSE on every machine. The disk figure will then report the storage volume rather than the data directory. How SeaweedFS's replication and erasure-coding options map onto the requirement "lose one disk or one machine without losing data" is to be settled in this phase. SeaweedFS supports several masters coordinated by Raft, which offers a path out of the head being a single point of failure for storage.